

Mini-PROJECT5 — AI Finance Approval Platform

Project Name: FinApprove AI — Enterprise Finance Approval Platform

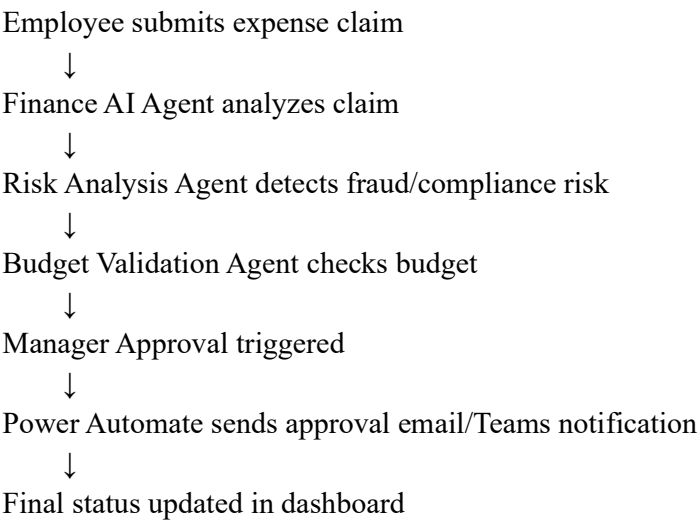
Problem Statement

In many enterprises, expense approvals are delayed because finance teams manually check:

- Expense amount
- Budget availability
- Policy violations
- Duplicate claims
- Fraud indicators
- Manager approval status
- Compliance risk

This project builds an **AI-powered finance approval web application** where employees submit expenses and AI agents validate, score risk, recommend approval/rejection, and trigger approval workflows.

1. Project Workflow



2. Core Features

Feature	Description
Expense Submission	Employee submits expense claim

Feature	Description
AI Expense Validation	AI checks policy, fraud, compliance
Budget Validation	Validates expense against department budget
Risk Scoring	Generates Low/Medium/High risk score
Approval Recommendation	Approve / Reject / Needs Review
Manager Approval	Trigger Power Automate approval
Dashboard	Shows all expense claims
Audit Trail	Stores AI reasoning and decisions

3. Tech Stack

Layer	Technology
Frontend	React
Backend	Node.js + Express
Database	MongoDB
AI	Azure OpenAI
Workflow	Power Automate
Notification	Outlook / Teams
Authentication	Entra ID optional

4. Folder Structure

finapprove-ai/

```

|
├── client/
│   └── src/
│       ├── App.js
│       ├── pages/
│       │   ├── ExpenseForm.js
│       │   └── Dashboard.js
│       └── services/

```

```
├── api.js
└── server/
    ├── server.js
    ├── .env
    ├── models/
    │   └── Expense.js
    ├── routes/
    │   └── expenseRoutes.js
    ├── controllers/
    │   └── expenseController.js
    └── agents/
        ├── financeAgent.js
        ├── riskAgent.js
        └── budgetAgent.js
```

5. Backend Setup

```
mkdir finapprove-ai
cd finapprove-ai
```

```
mkdir server
cd server
```

```
npm init -y
npm install express cors dotenv axios mongoose
```

server/.env

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/finapprove_ai
```

```
AZURE_OPENAI_ENDPOINT=https://YOUR-RESOURCE.openai.azure.com
AZURE_OPENAI_KEY=YOUR_AZURE_OPENAI_KEY
AZURE_OPENAI_DEPLOYMENT=YOUR_MODEL_DEPLOYMENT_NAME
AZURE_OPENAI_API_VERSION=2024-02-15-preview
```

```
POWER_AUTOMATE_WEBHOOK_URL=YOUR_POWER_AUTOMATE_HTTP_TRIGGER_URL
```

server/server.js

```
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
require("dotenv").config();

const expenseRoutes = require("./routes/expenseRoutes");

const app = express();

app.use(cors());
app.use(express.json());

app.use("/api/expenses", expenseRoutes);

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB connected"))
  .catch((error) => console.log("MongoDB error:", error.message));

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Finance Approval API running on port ${PORT}`);
});
```

6. MongoDB Model

server/models/Expense.js

```
const mongoose = require("mongoose");

const ExpenseSchema = new mongoose.Schema(
  {
    employeeName: String,
    employeeEmail: String,
    department: String,
    expenseType: String,
    amount: Number,
    description: String,

    aiRecommendation: String,
    riskLevel: String,
    riskScore: Number,
```

```

    policyStatus: String,
    budgetStatus: String,
    aiReasoning: String,

    approvalStatus: {
      type: String,
      default: "Pending"
    },

    managerEmail: String
  },
  {
    timestamps: true
  }
);

module.exports = mongoose.model("Expense", ExpenseSchema);

```

7. AI Agents

server/agents/financeAgent.js

```

const axios = require("axios");

async function analyzeExpense(expense) {
  const url =
    `${process.env.AZURE_OPENAI_ENDPOINT}/openai/deployments/` +
    `${process.env.AZURE_OPENAI_DEPLOYMENT}/chat/completions` +
    `?api-version=${process.env.AZURE_OPENAI_API_VERSION}`;

  const financePrompt = `
You are an enterprise Finance Approval AI Agent.

Analyze this expense claim.

Check:
- policy violations
- fraud indicators
- unusual amounts
- compliance risk
- approval recommendation

Return only valid JSON in this format:

```

```
{
  "aiRecommendation": "Approve | Reject | Needs Review",
  "riskLevel": "Low | Medium | High",
  "riskScore": 0,
  "policyStatus": "Compliant | Non-Compliant | Needs Review",
  "aiReasoning": "short explanation"
}
```

Expense Details:

Employee Name: \${expense.employeeName}

Department: \${expense.department}

Expense Type: \${expense.expenseType}

Amount: \${expense.amount}

Description: \${expense.description}

```
`;
```

```
const response = await axios.post(
  url,
  {
    messages: [
      {
        role: "system",
        content:
          "You are a strict enterprise finance compliance and approval AI agent."
      },
      {
        role: "user",
        content: financePrompt
      }
    ],
    temperature: 0.2
  },
  {
    headers: {
      "api-key": process.env.AZURE_OPENAI_KEY,
      "Content-Type": "application/json"
    }
  }
);
```

```
const content = response.data.choices[0].message.content;
```

```
    return JSON.parse(content);
}
```

```
module.exports = {
  analyzeExpense
};
```

server/agents/budgetAgent.js

```
function validateBudget(expense) {
  const departmentBudgets = {
    HR: 50000,
    IT: 150000,
    Finance: 100000,
    Sales: 200000,
    Marketing: 120000
  };

```

```
  const availableBudget = departmentBudgets[expense.department] || 50000;
```

```
  if (expense.amount > availableBudget) {
    return {
      budgetStatus: "Budget Exceeded",
      availableBudget
    };
  }

```

```
  return {
    budgetStatus: "Within Budget",
    availableBudget
  };
}
```

```
module.exports = {
  validateBudget
};
```

server/agents/riskAgent.js

```
function ruleBasedRiskCheck(expense) {
  let riskScore = 0;
  const risks = [];
```

```

if (expense.amount > 100000) {
  riskScore += 40;
  risks.push("High amount expense");
}

if (
  expense.description.toLowerCase().includes("cash") ||
  expense.description.toLowerCase().includes("urgent")
) {
  riskScore += 20;
  risks.push("Suspicious keywords found");
}

if (expense.expenseType === "Travel" && expense.amount > 75000) {
  riskScore += 20;
  risks.push("High travel expense");
}

let riskLevel = "Low";

if (riskScore >= 60) {
  riskLevel = "High";
} else if (riskScore >= 30) {
  riskLevel = "Medium";
}

return {
  ruleRiskScore: riskScore,
  ruleRiskLevel: riskLevel,
  ruleRisks: risks
};
}

module.exports = {
  ruleBasedRiskCheck
};

```

8. Power Automate Trigger Service

server/agents/approvalAgent.js


```

const axios = require("axios");

async function triggerApprovalWorkflow(expense) {
  if (!process.env.POWER_AUTOMATE_WEBHOOK_URL) {
    return {
      triggered: false,
      message: "Power Automate webhook not configured"
    };
  }

  const payload = {
    employeeName: expense.employeeName,
    employeeEmail: expense.employeeEmail,
    managerEmail: expense.managerEmail,
    department: expense.department,
    expenseType: expense.expenseType,
    amount: expense.amount,
    aiRecommendation: expense.aiRecommendation,
    riskLevel: expense.riskLevel,
    aiReasoning: expense.aiReasoning
  };

  await axios.post(process.env.POWER_AUTOMATE_WEBHOOK_URL, payload);

  return {
    triggered: true,
    message: "Approval workflow triggered"
  };
}

module.exports = {
  triggerApprovalWorkflow
};

```

9. Controller

server/controllers/expenseController.js

```

const Expense = require("../models/Expense");
const { analyzeExpense } = require("../agents/financeAgent");
const { validateBudget } = require("../agents/budgetAgent");
const { ruleBasedRiskCheck } = require("../agents/riskAgent");
const { triggerApprovalWorkflow } = require("../agents/approvalAgent");

```

```

exports.submitExpense = async (req, res) => {
  try {
    const expenseData = req.body;

    const aiResult = await analyzeExpense(expenseData);
    const budgetResult = validateBudget(expenseData);
    const ruleRisk = ruleBasedRiskCheck(expenseData);

    let finalRiskScore = Math.max(
      aiResult.riskScore || 0,
      ruleRisk.ruleRiskScore || 0
    );

    let finalRiskLevel = aiResult.riskLevel;

    if (finalRiskScore >= 60) {
      finalRiskLevel = "High";
    } else if (finalRiskScore >= 30) {
      finalRiskLevel = "Medium";
    } else {
      finalRiskLevel = "Low";
    }

    let approvalStatus = "Pending Manager Approval";

    if (
      aiResult.aiRecommendation === "Reject" ||
      budgetResult.budgetStatus === "Budget Exceeded"
    ) {
      approvalStatus = "Needs Finance Review";
    }

    const expense = await Expense.create({
      ...expenseData,
      aiRecommendation: aiResult.aiRecommendation,
      riskLevel: finalRiskLevel,
      riskScore: finalRiskScore,
      policyStatus: aiResult.policyStatus,
      budgetStatus: budgetResult.budgetStatus,
      aiReasoning: aiResult.aiReasoning,
      approvalStatus
    });
  }
}

```

```

    await triggerApprovalWorkflow(expense);

    res.status(201).json({
      message: "Expense submitted successfully",
      expense
    });
  } catch (error) {
    console.log(error);

    res.status(500).json({
      message: "Expense processing failed",
      error: error.message
    });
  }
};

exports.getExpenses = async (req, res) => {
  try {
    const expenses = await Expense.find().sort({ createdAt: -1 });

    res.json(expenses);
  } catch (error) {
    res.status(500).json({
      message: "Unable to fetch expenses",
      error: error.message
    });
  }
};

```

10. Routes

server/routes/expenseRoutes.js

```

const express = require("express");
const router = express.Router();

const {
  submitExpense,
  getExpenses
} = require("../controllers/expenseController");

router.post("/", submitExpense);

```

```
router.get("/", getExpenses);
```

```
module.exports = router;
```

11. Frontend Setup

```
cd ..
```

```
npx create-react-app client
```

```
cd client
```

```
npm install axios
```

client/src/services/api.js

```
import axios from "axios";
```

```
const API_BASE_URL = "http://localhost:5000/api";
```

```
export async function submitExpense(data) {  
  const response = await axios.post(`${API_BASE_URL}/expenses`, data);  
  return response.data;  
}
```

```
export async function getExpenses() {  
  const response = await axios.get(`${API_BASE_URL}/expenses`);  
  return response.data;  
}
```

client/src/App.js

```
import React from "react";
```

```
import ExpenseForm from "../pages/ExpenseForm";
```

```
import Dashboard from "../pages/Dashboard";
```

```
function App() {  
  return (  
    <div style={{ padding: "30px", fontFamily: "Arial" }}>  
      <h1>FinApprove AI</h1>  
      <p>Enterprise AI Finance Approval Platform</p>  
  
      <ExpenseForm />  
      <hr />  
      <Dashboard />  
    </div>  
  );  
}
```

```
    </div>
  );
}
```

```
export default App;
```

client/src/pages/ExpenseForm.js

```
import React, { useState } from "react";
import { submitExpense } from "../services/api";

function ExpenseForm() {
  const [form, setForm] = useState({
    employeeName: "",
    employeeEmail: "",
    department: "Finance",
    expenseType: "Travel",
    amount: "",
    description: "",
    managerEmail: ""
  });

  const [result, setResult] = useState(null);

  function handleChange(e) {
    setForm({
      ...form,
      [e.target.name]: e.target.value
    });
  }

  async function handleSubmit(e) {
    e.preventDefault();

    const payload = {
      ...form,
      amount: Number(form.amount)
    };

    const response = await submitExpense(payload);
    setResult(response.expense);
  }
}
```

```
return (  
  <div>  
    <h2>Submit Expense Claim</h2>  
  
    <form onSubmit={handleSubmit}>  
      <input  
        name="employeeName"  
        placeholder="Employee Name"  
        value={form.employeeName}  
        onChange={handleChange}  
      />  
      <br /><br />  
  
      <input  
        name="employeeEmail"  
        placeholder="Employee Email"  
        value={form.employeeEmail}  
        onChange={handleChange}  
      />  
      <br /><br />  
  
      <select  
        name="department"  
        value={form.department}  
        onChange={handleChange}  
      >  
        <option>Finance</option>  
        <option>HR</option>  
        <option>IT</option>  
        <option>Sales</option>  
        <option>Marketing</option>  
      </select>  
      <br /><br />  
  
      <select  
        name="expenseType"  
        value={form.expenseType}  
        onChange={handleChange}  
      >  
        <option>Travel</option>  
        <option>Food</option>  
        <option>Software</option>  
        <option>Training</option>
```

```
<option>Office Equipment</option>
</select>
<br /><br />
```

```
<input
  name="amount"
  type="number"
  placeholder="Amount"
  value={form.amount}
  onChange={handleChange}
/>
<br /><br />
```

```
<textarea
  name="description"
  placeholder="Expense Description"
  value={form.description}
  onChange={handleChange}
/>
<br /><br />
```

```
<input
  name="managerEmail"
  placeholder="Manager Email"
  value={form.managerEmail}
  onChange={handleChange}
/>
<br /><br />
```

```
<button type="submit">Submit for AI Approval</button>
</form>
```

```
{result && (
  <div style={{ marginTop: "20px", padding: "15px", border: "1px solid #ccc" }}>
    <h3>AI Decision</h3>
    <p><b>Recommendation:</b> {result.aiRecommendation}</p>
    <p><b>Risk Level:</b> {result.riskLevel}</p>
    <p><b>Risk Score:</b> {result.riskScore}</p>
    <p><b>Policy Status:</b> {result.policyStatus}</p>
    <p><b>Budget Status:</b> {result.budgetStatus}</p>
    <p><b>AI Reasoning:</b> {result.aiReasoning}</p>
    <p><b>Approval Status:</b> {result.approvalStatus}</p>
  </div>
```

```

    })
  </div>
);
}

export default ExpenseForm;

```

client/src/pages/Dashboard.js

```

import React, { useEffect, useState } from "react";
import { getExpenses } from "../services/api";

function Dashboard() {
  const [expenses, setExpenses] = useState([]);

  async function loadExpenses() {
    const data = await getExpenses();
    setExpenses(data);
  }

  useEffect(() => {
    loadExpenses();
  }, []);

  return (
    <div>
      <h2>Finance Approval Dashboard</h2>

      <button onClick={loadExpenses}>Refresh</button>

      <table border="1" cellPadding="10" style={{ marginTop: "20px", width: "100%" }}>
        <thead>
          <tr>
            <th>Employee</th>
            <th>Department</th>
            <th>Expense Type</th>
            <th>Amount</th>
            <th>AI Recommendation</th>
            <th>Risk</th>
            <th>Budget</th>
            <th>Status</th>
          </tr>
        </thead>

```



```

    <tbody>
      {expenses.map((expense) => (
        <tr key={expense._id}>
          <td>{expense.employeeName}</td>
          <td>{expense.department}</td>
          <td>{expense.expenseType}</td>
          <td>{expense.amount}</td>
          <td>{expense.aiRecommendation}</td>
          <td>{expense.riskLevel}</td>
          <td>{expense.budgetStatus}</td>
          <td>{expense.approvalStatus}</td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
);
}

```

```
export default Dashboard;
```

12. Run Project

Start MongoDB

```
mongod
```

Start Backend

```
cd server
node server.js
```

Start Frontend

```
cd client
npm start
```

13. Sample Input

```

{
  "employeeName": "Rajesh Kumar",
  "employeeEmail": "rajesh@company.com",
  "department": "Sales",
  "expenseType": "Travel",

```

```
"amount": 95000,  
"description": "Urgent client visit travel expense with hotel and local transport",  
"managerEmail": "manager@company.com"  
}
```

14. Expected AI Output

```
{  
  "aiRecommendation": "Needs Review",  
  "riskLevel": "Medium",  
  "riskScore": 45,  
  "policyStatus": "Needs Review",  
  "budgetStatus": "Within Budget",  
  "aiReasoning": "The claim amount is high for travel and requires manager validation with  
supporting bills.",  
  "approvalStatus": "Pending Manager Approval"  
}
```

15. Power Automate Demo Steps

Create a flow:

Trigger:

- When an HTTP request is received

Input JSON:

- employeeName
- employeeEmail
- managerEmail
- department
- expenseType
- amount
- aiRecommendation
- riskLevel
- aiReasoning

Action 1:

- Start and wait for an approval

Action 2:

- Send email to manager

Action 3:

- Post message in Teams

Action 4:

- Return approval status
-

16. Enterprise Conclusion

This mini project demonstrates:

- AI finance approval automation
- AI reasoning engine
- Risk scoring
- Budget validation
- Approval workflow automation
- Power Automate integration
- MongoDB persistence
- Enterprise dashboard
- AB-100 style business solution design